

Kev and CLM

Difference	Kev (small models)	CLM
Inputs	State, question and options together	State and actions encoded separately
Trainable	LoRA + pointer head	Projection heads only
Features	Encoder behavior changes	LLM features stay fixed

Our lab follows Kev's LoRA + pointer-head recipe.

Key and CLM: pros and cons

Criterion	Key	CLM
Strength	Adapts encoder features Small released models	Reuses action embeddings Cheap repeated head training
Tradeoff	Backpropagates through the encoder for LoRA	Depends on fixed features No joint token interaction
Lab fit	Main lab recipe Full base stage is public	Useful comparison Fixed-feature head training

Accuracy depends on the backbone, data and task.

Training cost

Work	Key LoRA	CLM heads
Encoder	Forward + backward during each training run	Extract fixed features once for each text
Repeat runs	Repeat encoder compute for every experiment	Reuse feature cache Train heads only
Budget	Encoder activations plus adapter optimizer	Feature extraction cost plus head optimizer

Compare total cost: feature extraction + all training runs.

Serving cost and model size

Factor	Kev	CLM
Each new state	Encode state and options Score pointer outputs	Encode state + question Score cached actions
Useful cache	Shared state prefix across questions	Independent action vectors across changing states
BF16 weights*	0.8B: ~1.6 GB 4B: ~8 GB	8B reference: ~16 GB Two paths can share weights

*Parameter count $\times$ 2 bytes; excludes heads, cache and runtime.

Performance evidence

Published result	Setup	What it supports
CLM: 36-44 ms	RTX 4090; cached actions 1-1,024 candidates	Low added cost as the action set grows
Kev-4B: 12 ms	RTX PRO 6000 One question; median	Fast serving in that llama.cpp setup
Kev vs CLM	No matched Pac-Man comparison established	Measure skill, latency and memory together

These reports do not establish a Kev-versus-CLM speed ranking.

Models supported by llama.cpp

Model	Readout	Useful strength	Main tradeoff
Julia-1 144M	Option-marker head	Small; multilingual 50+ languages	Limited knowledge/reasoning Private training pipeline
Laya 421M	Option-marker head	Small; designed for specialization	Fully fine-tuned encoder English root: 512 tokens
Kev-4B	Pointer head	Adaptable features Open training code	LoRA changes features Options encoded each step
lev 4B	Vocabulary label logits	Existing LM readout Many label codes	LoRA; two choice orders Extra head not converted
OpenJev 27B	Vocabulary label logits	Text and images Agent decisions	Larger model Non-commercial weights

How llama.cpp scores decisions

POST /v1/systemone: Choice, Score and Noul

GGUF metadata selects the model's readout

Kev's LoRA is merged before quantization

No generated answer tokens to parse

CLM needs its own serving implementation

How SGLang scores decisions

/v1/decisions and /v1/systemone are merged

Chat-model label logits; no generated tokens

One prompt per question; normal prefix caching

Checked: Qwen3.8-27B and Qwen3.5-35B-A3B

Native Intern-Decision support is an open PR

A fair Kev-versus-CLM comparison

Use the same boards, labels and legal actions

Time cold and warm caches on one device

Measure accuracy, captures, dots and latency

Record training time and peak memory

Match backbones to isolate the architecture

Our Kev-4B Pac-Man lab

Classic maze, power pellets and four ghost AIs

Skills checkpoint controls Pac-Man at CP0

Native-engine search generates task labels

Fine-tune the same LoRA and pointer head

Compare the model with its planning teacher

Our 90-minute route

0-10 Compare architectures and play

10-20 Inspect the four general training stages

20-30 Review Pac-Man player labels

30-60 Fine-tune LoRA and the pointer head

60-80 Compare cost and player decisions

80-90 Explain and submit

Colab runtime for this lab

Target: RTX PRO 6000 Blackwell, 96 GB

Locked PyTorch 2.8 / CUDA 12.8 and BF16

Required FLA, causal convolution, fused AdamW

Pac-Man: batch 4 × accumulation 2

Checkpointing on; row budget 0

CP0: Pac-Man and four ghost personalities

Blinky pursues; Pinky targets ahead

Inky uses Blinky; Clyde retreats when close

Ghosts leave home through counters/timeouts

Check each ghost mode and simulation seconds

Verify the active Skills adapter in the badge

CP1: score a legal player move

Pack state, question and legal options

Read decision-token and option-token features

Score each option with the pointer head

Read probabilities by direction name

Apply the move through the game code

What a probability tells us

Select the highest-scoring legal action

Inspect competing directions

Compare calibration on labelled examples

Confidence is not maze success

A legal move can still be strategically poor

CP2: inspect planning labels

4,096 train; 256 development; 256 evaluation

Simulate up to 20 moves in the native engine

Keep eight paths per first legal action

Prioritize survival, score, pellets and progress

Review labels; keep episodes in separate splits

Kev-4B training stages

- 1 Decisions: 12,576 records $\times$ 2 epochs
- 2 Dates/evidence: 1,425 + 2,000 replay
- 3 Documents: 5,219 + 2,000 replay
- 4 Skills/devtools: 11,320 + 4,000 replay
- 5 Pac-Man: 4,096 + 2,000 replay; 1 epoch

CP3: 30 minutes of fine-tuning

Warm-start your completed Skills checkpoint

One epoch; lr $2e-5$; effective batch eight

Train rank-16 LoRA and 256-dim pointer head

Keep none/distractor augmentation disabled

Save recovery and enabled Drive backups

Check what training changed

Trainable: LoRA and pointer-head parameters

Original base matrices remain fixed

Encoder features change through LoRA

Track loss and live memory in TensorBoard

Save recovery weights during every stage

CP4: compare the same boards

Fix Skills and planner-trained checkpoints

Score the same 256 evaluation boards

Compare strict and tie-aware teacher agreement

Inspect search regret and immediate captures

Save paired predictions by board ID

CP4: compare cost and gameplay

Use the same native engine and starting seed

Compare pellets, score and repeated tiles

Verify which LoRA is serving every decision

Record step time, GPU use and compute units

Inspect the five-seed CPU teacher comparison

CP5: explain and submit

Explain one changed move and one mistake

Show which parameters received gradients

Save all five adapters and pointer heads

Submit paired predictions and training curves

Record runtime and actual compute-unit use

The native game and planning teacher

Game: masonicGIT/pacman at 7407174

Original maze, renderer, sounds and mechanics

Predict each ghost with its native targeting code

Search uses current timers; model sees the board

Candidate paths stay outside the model input